

Mock Midterm

Q1. Conceptual / Short QA

(a)

What is a Merkle tree? Explain how it helps in Bitcoin block verification.

Answer:

A Merkle tree is a binary tree of hashes built from transaction hashes in a block. Each leaf node is the hash of a transaction, and each internal node is the hash of the concatenation of its two child hashes. The final top hash is called the Merkle root.

It helps in Bitcoin block verification because a node can prove that a transaction belongs to a block using only a small set of sibling hashes, called a Merkle proof, instead of downloading every transaction in the block. This makes verification more efficient in terms of storage and bandwidth.

Explanation:

The key points examiner wants:

- leaves are transaction hashes
 - parent = hash of children
 - top = Merkle root
 - efficient proof of inclusion
-

(b)

What happens if a transaction input refers to a UTXO that is not present in the UTXO database?

Answer:

If a transaction input refers to a UTXO that is not present in the UTXO database, the transaction is invalid. This means the output either does not exist or has already been spent.

Explanation:

The UTXO database contains all spendable outputs. If the referenced output is missing, the node cannot accept the input as valid.

Q2. Conceptual / Short QA

(a)

Explain the difference between transaction fee and fee rate.

Answer:

Transaction fee is the difference between the total value of the inputs and the total value of the outputs.

$\text{Fee} = \text{total inputs} - \text{total outputs}$

Fee rate is the fee per unit size of the transaction, usually measured in satoshis per byte or satoshis per vbyte.

$\text{Fee rate} = \text{fee} / \text{transaction size}$

Explanation:

A transaction may have a high fee but also be very large. Miners often prefer transactions with a higher fee rate because they earn more per unit of block space.

(b)

Why does a miner keep changing the nonce during proof-of-work?

Answer:

A miner keeps changing the nonce because each nonce produces a different block header hash. The miner is trying to find a hash that is smaller than the target. Since hash output is unpredictable, repeated nonce changes give repeated chances to find a valid hash.

Explanation:

This is the basic trial-and-error process of mining.

Q3. Simulation / Numerical

Merkle Tree Construction

Assume the transaction hashes are:

- $T1 = a1$
- $T2 = b2$
- $T3 = c3$
- $T4 = d4$

And define the hash function for this question as:

$$H(x \parallel y) = [x \ y]$$

This is a fake symbolic hash just for simulation.

Construct the Merkle tree and find the Merkle root.

Answer:

First level:

- $H12 = H(a1 \parallel b2) = [a1b2]$
- $H34 = H(c3 \parallel d4) = [c3d4]$

Root:

- $\text{Root} = H([a1b2] \parallel [c3d4]) = [[a1b2][c3d4]]$

So the Merkle root is:

$[[a1b2][c3d4]]$

Explanation:

This question checks whether you know the structure:

- pair leaves
- hash upward
- final top hash is root

Follow-up

If we want to prove T3 is in the block, what hashes are needed in the Merkle proof?

Answer:

To prove T3 is included, we need:

- sibling of T3, which is T4 = d4
- sibling of parent H34, which is H12 = [a1b2]

So proof elements are:

- d4
- [a1b2]

Explanation:

You need the path from leaf to root.

Q4. Simulation / Numerical

UTXO, unlocking, fee

A transaction has the following inputs and outputs:

Inputs

- Input 1 spends UTXO worth 1.20 BTC
- Input 2 spends UTXO worth 0.80 BTC

Outputs

- Output 1 = 1.50 BTC
- Output 2 = 0.40 BTC

(a) Calculate the transaction fee.

Answer:

Total inputs = $1.20 + 0.80 = 2.00$ BTC

Total outputs = $1.50 + 0.40 = 1.90$ BTC

Fee = $2.00 - 1.90 = \mathbf{0.10\ BTC}$

Explanation:

Always do:

fee = inputs – outputs

(b) If the transaction size is 250 bytes, calculate the fee rate in BTC per byte.

Answer:

Fee rate = $0.10 / 250 = 0.0004$ BTC per byte

Explanation:

Just divide total fee by size.

(c) After this transaction is accepted, what happens to the UTXO database?

Answer:

The two old UTXOs referenced by Input 1 and Input 2 are removed from the UTXO database because they have now been spent.

The two new outputs created by this transaction:

- 1.50 BTC
- 0.40 BTC

are added to the UTXO database as new spendable outputs.

Explanation:

This is the most important UTXO update rule:

- old referenced outputs removed
 - new outputs added
-

Q5. Simulation / Numerical

PoW / target / nonce

Assume a block is valid only if:

$$\text{hash}(\text{block header}) < 5000$$

A miner tries different nonces and gets the following hash outputs:

Nonce	Hash output
-------	-------------

21	8123
----	------

22	7450
----	------

23	4998
----	------

24	5301
----	------

(a) Which nonce produces a valid block?

Answer:

A valid block needs hash < 5000 .

Check values:

- 8123 $\rightarrow$ invalid
- 7450 $\rightarrow$ invalid
- 4998 $\rightarrow$ valid
- 5301 $\rightarrow$ invalid

So the valid nonce is **23**.

Explanation:

Only 4998 is less than 5000.

(b) Why are the other nonces invalid?

Answer:

They are invalid because their hash outputs are greater than or equal to the target threshold. In proof-of-work, only hashes smaller than the target are accepted.

Explanation:

PoW is not about getting any hash. It must be below target.

(c) If the target becomes 3000 instead of 5000, would nonce 23 still work?

Answer:

No.

Nonce 23 gives hash 4998, and 4998 is not less than 3000.

So nonce 23 would become invalid.

Explanation:

Smaller target means harder difficulty.

Q6. Objective / Fill the Table

Fill the following table:

Type	Full form		Main unlocking/spending requirement
P2PKH	?	?	
P2MS	?	?	
P2SH	?	?	
P2WPKH	?	?	
P2WSH	?	?	
P2TR	?	?	

Answer:

Type	Full form	Main unlocking/spending requirement
P2PKH	Pay to Public Key Hash	valid signature + matching public key

P2MS	Pay to Multisig	required number of valid signatures
P2SH	Pay to Script Hash	redeem script + data satisfying that script
P2WPKH	Pay to Witness Public Key Hash	witness signature + public key
P2WSH	Pay to Witness Script Hash	witness script + required witness data
P2TR	Pay to Taproot	taproot key-path or script-path satisfaction

Explanation:

This is pure identification/matching. Very likely in your paper because sir said one question may be a partially filled table.

Extra Practice Questions

1. Signature logic

A P2PKH output is being spent. The spender provides:

- a public key whose hash matches the stored hash
- but the signature is invalid

Will the spend succeed?

Answer:

No. The spend fails because both conditions are needed:

- correct public key
 - valid signature
-

2. Merkle proof logic

A computed Merkle root from the proof path is different from the block's Merkle root. What does that show?

Answer:

It shows the transaction inclusion proof failed. The transaction is not proven to belong to that block.

3. UTXO logic

Can one UTXO be spent twice in two valid confirmed transactions?

Answer:

No. Once a UTXO is spent, it is removed from the UTXO set. Any second attempt is a double-spend and will be invalid.

Super compact one-page cram list

Memorize this the night before:

Formulas

- **Fee = inputs – outputs**
- **Fee rate = fee / size**
- **PoW valid if hash < target**

Merkle

- leaves = tx hashes
- parent = hash(left || right)
- root = top hash
- proof = sibling hashes + order

UTXO

- inputs consume old UTXOs
- outputs create new UTXOs
- missing UTXO = invalid
- already spent = double-spend

Signature

- sign with private key
- verify with public key
- invalid signature = reject

Types

- P2PKH = sig + pubkey
- P2MS = multiple sigs
- P2SH = redeem script + data
- P2WPKH = SegWit pubkey hash
- P2WSH = SegWit script hash
- P2TR = Taproot